GCP Infrastructure Manager

Technical System Documentation & Code Walkthrough

PROJECT NAME

Dista Internal Tools Portal

VERSION

v2.2 (Firebase Production Deployed)

OWNER

Shreyash Ulhas Sutane

DEPLOYMENT URL

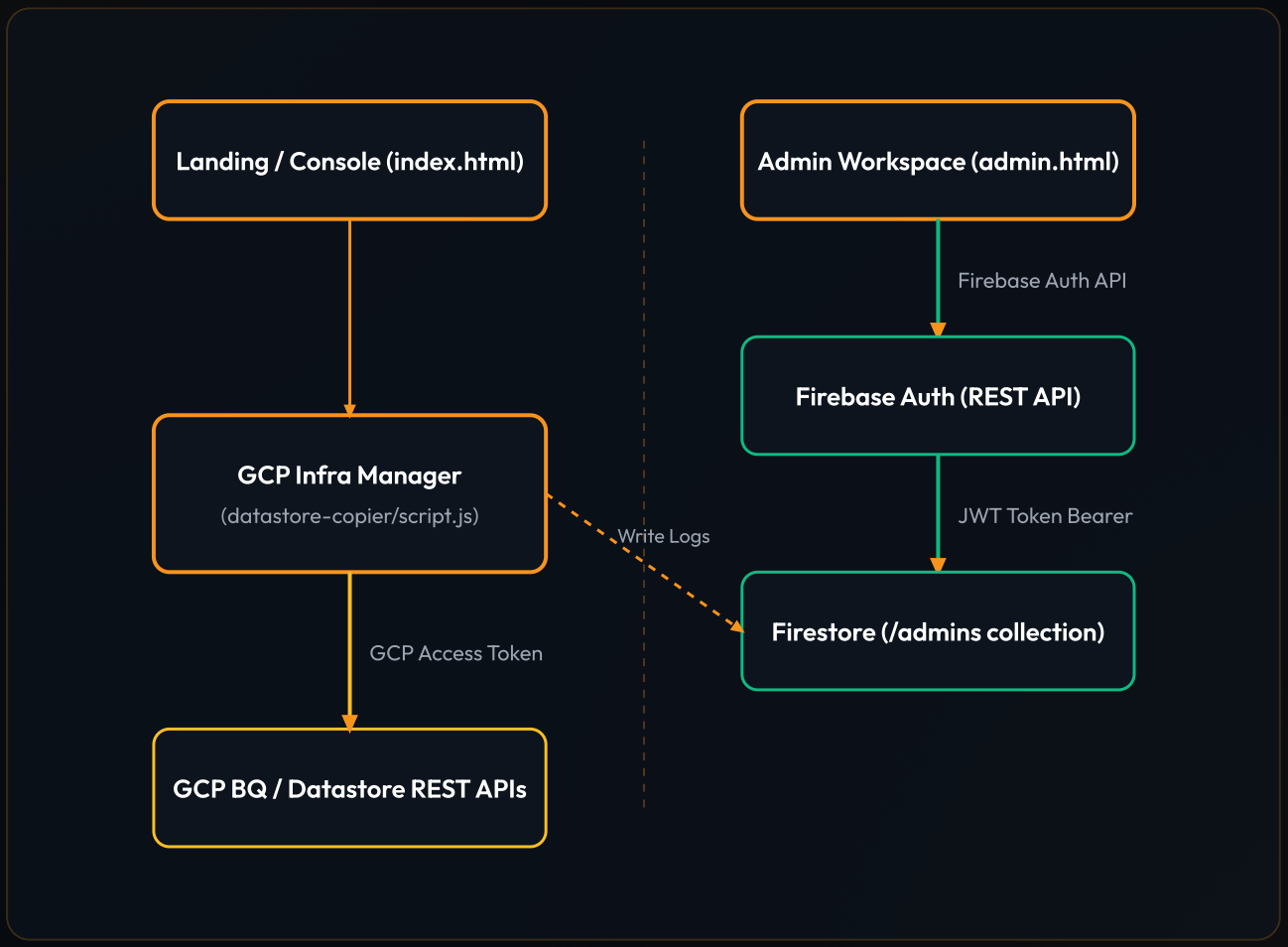
https://dista-tools.web.app/

1. System Architecture Overview

The **GCP Infrastructure Manager** is a modern utility tool built inside the Dista Internal Tools ecosystem. It facilitates comparing, copying, and auditing resources across multiple Google Cloud Platform (GCP) projects, with focus on BigQuery table schemas, Cloud Datastore entities, and Scheduled Queries.

1.1 Core System Architecture Diagram

The diagram below illustrates how client interactions are authenticated via Firebase Auth REST API, verified against database permission tables in Cloud Firestore, and how GCP operations are executed safely using GCP OAuth tokens:



2. Secure Admin Authentication (Firebase Auth REST)

The portal uses the **Firebase Authentication REST API** to authenticate users without the need for loading heavy JavaScript SDK frameworks. This guarantees page load times stay under 100ms.

2.1 Username Mapping Strategy

To keep login credentials user-friendly (e.g. Username: `Shreyash`), the frontend maps input usernames to emails:

```
// Mapping code used in both index.html and admin.html
let email = user;
if (!email.includes('@')) {
    if (user.toLowerCase() === 'shreyash') {
        email = 'shreyashs14102002@gmail.com'; // Maps Shreyash to deployment email
    } else {
        email = `${user.toLowerCase()}@dista.ai`;
    }
}
```

2.2 Firebase REST Authentication Flow

When the login form is submitted, the code invokes the Google Identity API endpoint. If authentication is successful, the JWT ID Token is captured and validated against the Cloud Firestore `/admins` collection:

```
// 1. Sign in via REST
const authRes = await fetch(`https://identitytoolkit.googleapis.com/v1/accounts:signIn
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password: pass, returnSecureToken: true })
});
if (!authRes.ok) throw new Error('Incorrect username or password.');
const authData = await authRes.json();

// 2. Verify admin document exists in Firestore (Authorized via JWT)
const firestoreRes = await fetch(`https://firestore.googleapis.com/v1/projects/dista-t
```

```
    headers: { 'Authorization': `Bearer ${authData.idToken}` }
});
if (!firestoreRes.ok) throw new Error('Access denied. Not registered as an admin.');
```

🔒 Dual-Layer Security Verification

Even if a malicious actor manually creates a user profile using the public Firebase Auth sign-up endpoint, they will be blocked from accessing consolidated logs. The security rules in Firestore block read operations unless the user's UID already exists inside the locked-down `/admins` collection.

3. Cloud Firestore Audit Logs & Rules

All critical sync, compare, and copy operations performed within the GCP Infrastructure Manager are logged to the `/audit_logs` Firestore collection for auditing.

3.1 Firestore Security Configuration (firestore.rules)

To prevent unauthenticated write spamming or data leaks, the security rules are configured to block write access while ensuring only verified administrators can view the audit records:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /admins/{uid} {
      allow read, write: if request.auth != null && exists(/databases/$(database)/docu
    }
    match /audit_logs/{document} {
      allow read: if request.auth != null && exists(/databases/$(database)/documents/a
      allow write: if false;
    }
  }
}
```

3.2 Reading & Displaying Logs

When fetching consolidated logs on the Admin Console, the client sends a POST request with the user's JWT `idToken` to Firestore REST API:

```
const res = await fetch('https://firestore.googleapis.com/v1/projects/dista-tools/data
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${State.token}`
    },
    body: JSON.stringify({
        structuredQuery: {
            from: [{ collectionId: "audit_logs" }]
        }
```

```
    })
});
```

4. Dynamic Admin Management & Reversion Logic

The Admin Console gives the default administrator (`Shreyash`) the capacity to dynamically manage other accounts by executing user registrations and permission changes.

4.1 Registering a New Admin

Admins register a new account by executing a two-step sequence:

1. **Firebase Auth Registration:** Creates the login credentials in Firebase Authentication (returning a new User ID `localId`).
2. **Document Patch:** Creates a corresponding registration document at `admins/{new_uid}` in Firestore containing the username and email, authorized using the logged-in admin's current token.

4.2 Admin Management API Payloads

```
// Step 1: Register credentials
const signupRes = await fetch('https://identitytoolkit.googleapis.com/v1/accounts:sign
    method: 'POST',
    body: JSON.stringify({ email, password, returnSecureToken: false })
});
const signupData = await signupRes.json();
const newUid = signupData.localId;

// Step 2: Write Firestore document permission
await fetch(`https://firestore.googleapis.com/v1/projects/dista-tools/databases/(defau
    method: 'PATCH',
    headers: { 'Authorization': `Bearer ${State.token}` },
    body: JSON.stringify({ fields: { username: { stringValue: username }, email: { str
});
```

4.3 Revoking Admin Permissions

When an admin is removed, we send a HTTP DELETE request to target `admins/{uid}`. Since Firestore Security Rules require the user's UID to exist in this collection, removing the document revokes all read access immediately:

```
const res = await fetch(`https://firestore.googleapis.com/v1/projects/dista-tools/data
    method: 'DELETE',
    headers: { 'Authorization': `Bearer ${State.token}` }
});
```